Université Moulay Ismail
Faculté des Sciences Meknès

Département d'Informatique

Rapport de Projet

Résolution du Problème du Voyageur de Commerce (TSP) par Optimisation par Essaim Particulaire (PSO)

Réalisé par :

RADY HASSANE

Année Universitaire : 2025 - 2026

Table des matières

Introduction Générale **2**

1 Définition et Formulation du Problème **3**
1.1 Présentation du TSP 3
1.2 Formulation Mathématique 3
1.3 Complexité du Problème 3

2 Méthodologies et Variantes de Résolution **4**
2.1 Algorithme 1 : L'Heuristique Gloutonne (Nearest Neighbor) 4
2.2 Algorithme 2 : PSO "Cold Start" (Initialisation Aléatoire) 5
2.3 Algorithme 3 : PSO "Warm Start" (Initialisation Heuristique) 6
2.4 Algorithme 4 : PSO Hybride Amélioré (PSO + 2-opt + Restart) 7
2.4.1 Mécanisme 1 : Hybridation avec 2-opt 7
2.4.2 Mécanisme 2 : Stratégie de "Restart" (Anti-stagnation) 7
2.4.3 Synthèse du fonctionnement de l'Algorithme 4 7

3 Implémentation et Résultats Expérimentaux **9**
3.1 Environnement de Test et Données 9
3.1.1 Environnement Technique 9
3.1.2 Description du Benchmark (TSPLIB) 9
3.2 Implémentation avec python 10
3.2.1 Bloc 1 : Importation et Chargement des Données 10
3.2.2 Bloc 2 : Cœur Algorithmique (Glouton, 2-opt, PSO) 11
3.2.3 Bloc 3 : Pipeline d'Exécution et Tests Unitaires 14
3.2.4 Bloc 4 : Agrégation et Visualisation Comparative 16
3.3 Analyse Détaillée : Étude de Cas Visuelle 17
3.3.1 Interprétation des résultats 18
3.4 Synthèse Globale et Analyse Comparative 18
3.4.1 Analyse détaillée des performances 19

Conclusion Générale **22**

Introduction Générale

L'optimisation combinatoire occupe une place centrale dans la recherche opérationnelle moderne. Elle vise à identifier la solution optimale au sein d'un ensemble discret et souvent massif de configurations possibles. Parmi les problèmes les plus emblématiques de ce domaine figure le **Problème du Voyageur de Commerce (Traveling Salesman Problem - TSP)**.

Le défi majeur du TSP réside dans sa complexité computationnelle. Étant classé comme un problème NP-difficile, sa résolution exacte devient impraticable pour des instances de grande taille. C'est dans ce contexte que les métaheuristiques, et plus particulièrement l'Optimisation par Essaim Particulaire (PSO), interviennent comme des alternatives performantes.

Ce projet se propose d'explorer et de mettre en œuvre une variante hybride du PSO pour résoudre efficacement le TSP, en comparant ses performances avec une approche gloutonne classique sur diverses instances de référence.

Chapitre 1

Définition et Formulation du Problème

1.1 Présentation du TSP

Le Problème du Voyageur de Commerce consiste à déterminer le cycle le plus court passant exactement une fois par chaque ville d'un ensemble donné et revenant à la ville de départ.

1.2 Formulation Mathématique

Le TSP peut être modélisé à l'aide d'un graphe complet $G = (V, E)$, où :

— $V = \{v_1, v_2, \ldots, v_n\}$ est l'ensemble des villes (sommets).

— $E = \{(i, j) : i, j \in V\}$ est l'ensemble des liaisons possibles (arêtes).

— Chaque arête (i, j) est associée à une distance $d_{ij} \geq 0$.

L'objectif est de trouver une permutation σ de $\{1, 2, \ldots, n\}$ qui minimise la fonction de coût suivante :

$$L = \sum_{i=1}^{n-1} d_{\sigma(i),\sigma(i+1)} + d_{\sigma(n),\sigma(1)}$$

1.3 Complexité du Problème

Le nombre total de circuits possibles pour un problème à n villes est de $(n-1)!/2$. Cette croissance factorielle illustre l'**explosion combinatoire** :

— Pour 10 villes : 181 440 circuits.

— Pour 20 villes : environ 6×10^{16} circuits.

Il est donc impossible d'utiliser une recherche exhaustive pour des instances réelles, ce qui justifie l'utilisation de méthodes approchées.

Chapitre 2

Méthodologies et Variantes de Résolution

Pour évaluer l'efficacité de notre approche, nous avons implémenté et comparé quatre configurations algorithmiques, allant de la simple heuristique à une métaheuristique hybride complexe.

2.1 Algorithme 1 : L'Heuristique Gloutonne (Nearest Neighbor)

Cette méthode sert de ligne de base (*baseline*).

— **Fonctionnement :** À chaque étape, l'algorithme choisit la ville non visitée la plus proche de la position actuelle.

— **Avantage :** Vitesse d'exécution quasi-instantanée.

— **Inconvénient :** Ne revient jamais sur ses décisions, ce qui produit souvent des trajectoires sous-optimales avec des "sauts" finaux très coûteux.

Algorithm 1 Algorithme 1 : Heuristique Gloutonne (Nearest Neighbor)

Entrée: Matrice des distances D, Ville de départ v_{start}

Sortie: Tour complet T, Coût total C

```
T ← [v_start]
Visites ← {v_start}
v_curr ← v_start
while |Visites| < N do
    v_next ← arg min_{v∉Visites}{D(v_curr, v)} {Ville la plus proche}
    Ajouter v_next à T
    Ajouter v_next à Visites
    v_curr ← v_next
end while
Ajouter v_start à la fin de T pour fermer la boucle
return T, CalculerCoût(T)
```

2.2 Algorithme 2 : PSO "Cold Start" (Initialisation Aléatoire)

C'est la version standard de l'Optimisation par Essaim Particulaire appliquée au TSP.

- **Fonctionnement :** L'essaim commence avec des particules dont les positions (tours) sont générées de manière totalement aléatoire.
- **Philosophie :** On compte uniquement sur la communication entre les particules (influence de *Pbest* et *Gbest*).
- **Comportement :** La convergence est souvent lente car l'espace de recherche est immense ($N!$).

Algorithm 2 Algorithme 2 : PSO Standard (Cold Start)

Entrée: Nombre de particules Pop, Itérations $MaxIter$

```
Initialiser Pop particules avec des positions Aléatoires
Calculer Pbest pour chaque particule
Identifier Gbest (Meilleur de tout l'essaim)
for t = 1 à MaxIter do
    for chaque particule P_i do
        S_p ← Séquence de swaps vers Pbest_i
        S_g ← Séquence de swaps vers Gbest
        Appliquer une combinaison de S_p et S_g sur P_i
        Évaluer nouveau coût de P_i
        if Nouveau coût < Pbest_i then
            Pbest_i ← P_i
        end if
    end for
    Mettre à jour Gbest si une particule a trouvé mieux
end for
return Gbest
```

2.3 Algorithme 3 : PSO "Warm Start" (Initialisation Heuristique)

Cette variante introduit une première forme d'intelligence collective dès le départ.

- **Fonctionnement :** Une particule de l'essaim est initialisée avec le résultat de l'algorithme **Glouton**.
- **Effet "Graine" :** Le *Gbest* initial est déjà de bonne qualité. Cela force les autres particules à explorer les zones proches de cette solution prometteuse.
- **Objectif :** Accélérer la convergence et garantir un résultat au moins égal au Glouton.

Algorithm 3 Algorithme 3 : PSO Warm Start (Avec Injection)

Entrée: Résultat du Glouton $T_{glouton}$

```
Particule_0 ← T_glouton {Injection de la solution gloutonne}
Initialiser les autres Particules_{1...N} Aléatoirement
Gbest ← Particule_0 {Le leader démarre fort}
for t = 1 à MaxIter do
    {Même logique de mouvement que PSO Standard}
    Calculer les vélocités (Swaps) vers Pbest et Gbest
    Mettre à jour les positions, scores et Gbest
end for
return Gbest
```

2.4 Algorithme 4 : PSO Hybride Amélioré (PSO + 2-opt + Restart)

C'est notre contribution la plus avancée. Elle combine trois mécanismes pour maximiser la performance.

2.4.1 Mécanisme 1 : Hybridation avec 2-opt

À chaque amélioration du meilleur score global (*Gbest*), nous appliquons une recherche locale **2-opt**. Le 2-opt "nettoie" les croisements d'arêtes que le PSO n'arrive pas à résoudre seul par de simples échanges.

2.4.2 Mécanisme 2 : Stratégie de "Restart" (Anti-stagnation)

Si la solution *Gbest* n'évolue plus pendant un certain nombre d'itérations (stagnation), une partie de l'essaim est réinitialisée aléatoirement.

2.4.3 Synthèse du fonctionnement de l'Algorithme 4

L'algorithme suit le cycle suivant :

$$\text{Exploration (PSO)} \rightarrow \text{Optimisation (2-opt)} \rightarrow \text{Vérification (Stagnation ?)} \rightarrow \text{Diversification (Restart)} \tag{2.1}$$

Algorithm 4 Algorithme 4 : PSO Hybride (Warm + 2-opt + Restart)

```
Initialiser avec Warm Start (Voir Algo 3)
Stagnation ← 0
for t = 1 à MaxIter do
    Exécuter mouvement PSO classique
    if Gbest a été amélioré then
        Stagnation ← 0
        Gbest ← Apply2Opt(Gbest) {Recherche Locale}
    else
        Stagnation ← Stagnation + 1
    end if
    if Stagnation > Seuil_limit then
        Restart Partiel : Réinitialiser 50% des particules les moins bonnes
        Stagnation ← 0
    end if
end for
return Gbest optimisé
```

Chapitre 3

Implémentation et Résultats Expérimentaux

Après avoir détaillé les fondements théoriques et les algorithmes développés, ce chapitre se consacre à la validation expérimentale. L'objectif est double : d'une part, vérifier la correction de notre implémentation, et d'autre part, quantifier les gains de performance apportés par l'hybridation du PSO face à une approche gloutonne classique.

Nous présenterons d'abord l'environnement de développement et les jeux de données utilisés, avant d'analyser les résultats obtenus en termes de qualité de solution (coût) et de temps de calcul.

3.1 Environnement de Test et Données

3.1.1 Environnement Technique

L'ensemble des algorithmes a été implémenté en langage **Python**, en utilisant les bibliothèques standards pour le calcul matriciel (*NumPy*) et la visualisation (*Matplotlib*). Les tests ont été exécutés sur la plateforme **Google Colab**, offrant un environnement standardisé.

3.1.2 Description du Benchmark (TSPLIB)

Pour garantir la fiabilité de nos comparaisons, nous avons utilisé des instances provenant de la **TSPLIB**, une bibliothèque de référence mondiale pour le problème du voyageur de commerce.

Nous avons sélectionné un panel de **14 instances** aux caractéristiques variées, allant de petites configurations (moins de 100 villes) à des problèmes de grande taille (plus de 2000 villes). Cette diversité nous permet de tester la **scalabilité** (passage à l'échelle) de notre algorithme PSO Hybride.

Voici la liste des instances utilisées, classées par ordre croissant de complexité :

Nom de l'instance	Nombre de Villes	Type de difficulté
`att48.tsp`	48	Faible (Validation)
`berlin52.tsp`	52	Faible
`gr96.tsp`	96	Faible
`rd100.tsp`	100	Moyenne
`bier127.tsp`	127	Moyenne
`si175.tsp`	175	Moyenne
`pr226.tsp`	226	Moyenne
`a280.tsp`	280	Moyenne
`gr431.tsp`	431	Forte
`d657.tsp`	657	Forte
`pr1002.tsp`	1002	Très Forte
`u1060.tsp`	1060	Très Forte
`rl1304.tsp`	1304	Très Forte
`u2152.tsp`	2152	Extrême (Stress Test)

Table 3.1 – Liste des instances TSPLIB utilisées pour les tests

Ce choix d'instances permet de couvrir différents types de géométries (répartitions uniformes, villes groupées en clusters, etc.) et de mesurer jusqu'à quel point le PSO Hybride parvient à optimiser le chemin lorsque l'explosion combinatoire devient critique (comme pour `u2152`).

3.2 Implémentation avec python

L'implémentation a été structurée de manière modulaire en quatre blocs principaux, allant du chargement des données à la visualisation finale. Cette section détaille chaque module en s'appuyant sur des extraits du code source.

3.2.1 Bloc 1 : Importation et Chargement des Données

Le premier module (Figure 3.1) gère l'environnement et l'interface avec les fichiers TSPLIB. Nous utilisons les bibliothèques scientifiques standards :

— **NumPy** : Pour les calculs matriciels rapides (distances, tableaux).

— **Pandas** : Pour structurer les résultats finaux.

— **Matplotlib** : Pour la génération des graphiques.

La fonction `load_instance_specific` est critique : elle parse les fichiers '.tsp', ignore les métadonnées textuelles et extrait uniquement les coordonnées (x, y) situées après la balise `NODE_COORD_SECTION`.

```python
import numpy as np
import pandas as pd
import time
import random
import matplotlib.pyplot as plt
from scipy.spatial import distance_matrix
from IPython.display import display
FILES_TO_TEST = [
    'att48.tsp',
    'a280.tsp',
    'berlin52.tsp',
    'bier127.tsp',
    'd657.tsp',
    'gr431.tsp',
    'gr96.tsp',
    'pr1002.tsp',
    'pr226.tsp',
    'rd100.tsp',
    'rl1304.tsp',
    'si175.tsp' ,
    'u1060.tsp',
    'u2152.tsp'
]

# Fonction de chargement universelle
def load_instance_specific(filepath):
    coords = []
    reading_coords = False
    with open(filepath, 'r') as f:
        for line in f:
            line = line.strip()
            if line == 'EOF': break
            if 'NODE_COORD_SECTION' in line:
                reading_coords = True
                continue
            if reading_coords:
                parts = line.split()
                if len(parts) >= 3:
                    try:
                        coords.append([float(parts[1]), float(parts[2])])
                    except ValueError:
                        continue
    return np.array(coords)
print("✅ BLOC 1 CHARGÉ : Prêt à lire les fichiers.")
```

```
✅ BLOC 1 CHARGÉ : Prêt à lire les fichiers.
```

FIGURE 3.1 – Module d'importation et fonction de parsing des fichiers TSPLIB

3.2.2 Bloc 2 : Cœur Algorithmique (Glouton, 2-opt, PSO)

Ce bloc (Figure 3.2) contient toute la logique mathématique décrite au Chapitre 2.

— **Fonctions utilitaires :** `calculate_cost` calcule la distance totale d'un tour, et `get_swap_sequence` traduit la différence entre deux tours en une liste d'échanges.
— **Heuristique Gloutonne :** La fonction `nearest_neighbor_tsp` implémente l'approche constructive rapide.
— **Recherche Locale :** La fonction `two_opt_local_search` optimise un tour en décroisant les arêtes. Notez l'optimisation : on ne recalcule pas tout le tour, mais seulement le delta des arêtes modifiées.
— **Classe Particle :** Le PSO est implémenté en Orienté Objet. Chaque particule mémorise sa meilleure position personnelle (`pbest`).

```python
def calculate_cost(tour, matrix):
    return matrix[tour, np.roll(tour, -1)].sum()

def get_swap_sequence(curr, target, limit=4):
    swaps = []
    temp = curr[:]
    for i in range(len(temp)):
        if temp[i] != target[i]:
            val = target[i]
            if val in temp:
                idx = temp.index(val)
                temp[i], temp[idx] = temp[idx], temp[i]
                swaps.append((i, idx))
                if len(swaps) >= limit: break
    return swaps

def apply_swaps(tour, swaps):
    new = tour[:]
    for i, j in swaps:
        new[i], new[j] = new[j], new[i]
    return new

# --- Algorithme Glouton ---
def nearest_neighbor_tsp(matrix, start_node=0):
    t0 = time.time()
    n = len(matrix)
    visited = np.zeros(n, dtype=bool)
    tour = [start_node]
    visited[start_node] = True
    curr = start_node
    for _ in range(n - 1):
        dists = matrix[curr].copy()
        dists[visited] = np.inf
        next_city = np.argmin(dists)
        tour.append(next_city)
        visited[next_city] = True
        curr = next_city
    return tour, calculate_cost(tour, matrix), time.time() - t0

# --- Recherche Locale (2-opt) ---
def two_opt_local_search(tour, matrix, max_passes=2):
    best_tour = np.array(tour)
    best_cost = calculate_cost(best_tour, matrix)
    improved = True
    passes = 0
    n = len(tour)
    window = 100 if n > 500 else n
    while improved and passes < max_passes:
        improved = False
        passes += 1
        for i in range(1, n - 2):
            limit_j = min(i + window, n)
            for j in range(i + 1, limit_j):
                if j - i == 1: continue
                old_edges = matrix[best_tour[i-1], best_tour[i]] + matrix[best_tour[j-1], best_tour[j]]
                new_edges = matrix[best_tour[i-1], best_tour[j-1]] + matrix[best_tour[i], best_tour[j]]
                if new_edges < old_edges:
                    best_tour[i:j] = best_tour[i:j][::-1]
                    best_cost -= (old_edges - new_edges)
                    improved = True
    return best_tour.tolist(), best_cost

# --- Classe Particule (PSO) ---
class Particle:
    def __init__(self, n, matrix, init_tour=None):
        self.position = list(np.random.permutation(n)) if init_tour is None else init_tour[:]
        self.cost = calculate_cost(self.position, matrix)
        self.pbest_pos = self.position[:]
        self.pbest_cost = self.cost

# --- PSO Principal ---
def run_pso_full(matrix, mode='cold', glouton_tour=None, iterations=100, num_particles=30):
    start_time = time.time()
    n = len(matrix)
    particles = []
    history = []

    for i in range(num_particles):
        if i == 0 and mode != 'cold' and glouton_tour:
            p = Particle(n, matrix, glouton_tour)
        else:
            p = Particle(n, matrix)
        particles.append(p)
```

3.2.3 Bloc 3 : Pipeline d'Exécution et Tests Unitaires

Le troisième module (Figure 3.3) orchestre la campagne de tests. Pour chaque fichier de la liste `FILES_TO_TEST`, le script exécute séquentiellement les quatre approches :

1. **Glouton** (Baseline).
2. **PSO Cold** (Initialisation aléatoire).
3. **PSO Warm** (Injection du résultat Glouton).
4. **PSO Amélioré** (Warm + plus d'itérations + Hybridation).

Ce bloc génère également les graphiques de convergence individuels pour visualiser la vitesse d'apprentissage de l'essaim sur chaque instance.

```python
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# Liste pour stocker le résumé de chaque fichier
global_stats = []

print(f"--- 🚀 DÉMARRAGE DE LA CAMPAGNE COMPLÈTE SUR {len(FILES_TO_TEST)} FICHIERS ---\n")

for filename in FILES_TO_TEST:
    print(f"\n{'='*80}")
    print(f"📂 TRAITEMENT : {filename}")
    print(f"{'='*80}")

    # 1. Chargement
    try:
        coords = load_instance_specific(filename)
        if len(coords) == 0:
            print(f"⚠️ Fichier vide ou illisible : {filename}")
            continue
        dist_matrix = distance_matrix(coords, coords)
    except Exception as e:
        print(f"❌ Erreur critique sur {filename}: {e}")
        continue
    N_VILLES = len(dist_matrix)
    print(f"📊 Taille : {N_VILLES} villes")
    ITER = 150 if N_VILLES < 200 else 80
    POP = 30

    # --- A) GLOUTON ---
    print("    🔹 1. Glouton...", end="\r")
    tour_g, cost_g, time_g = nearest_neighbor_tsp(dist_matrix)
    print(f"    ✅ 1. Glouton     : {cost_g:.0f} (ref)")

    print("    🔹 2. PSO Cold...", end="\r")
    min_c, _, hist_c, t_c, _ = run_pso_full(dist_matrix, 'cold', iterations=ITER, num_particles=POP)
    print(f"    ✅ 2. PSO Cold    : {min_c:.0f}")

    # --- C) PSO WARM (Start = Glouton) ---
    print("    🔹 3. PSO Warm...", end="\r")
    min_w, _, hist_w, t_w, _ = run_pso_full(dist_matrix, 'warm', glouton_tour=tour_g, iterations=ITER, num_particles=POP)
    print(f"    ✅ 3. PSO Warm    : {min_w:.0f}")

    print("    🔹 4. PSO Amélioré...", end="\r")
    min_i, _, hist_i, t_i, best_tour = run_pso_full(dist_matrix, 'improved', glouton_tour=tour_g, iterations=ITER+50, num_particles=POP)
    print(f"    ✅ 4. PSO Amélioré: {min_i:.0f} 🏆")

    gain_pct = ((cost_g - min_i) / cost_g) * 100

    # --- STOCKAGE ---
    global_stats.append({
        "Instance": filename,
        "Villes": N_VILLES,
        "Glouton": cost_g,
        "PSO Cold": min_c,
        "PSO Warm": min_w,
        "PSO Imp.": min_i,
        "Gain Final (%)": gain_pct,
        "Temps (s)": t_i # Temps du dernier algo (le plus long)
    })

    plt.figure(figsize=(12, 4))

    plt.subplot(1, 2, 1)
    plt.plot(hist_c, label='Cold', color='gray', linestyle=':', alpha=0.6)
    plt.plot(hist_w, label='Warm', color='orange', linestyle='--')
    plt.plot(hist_i, label='Amélioré', color='green', linewidth=2)
    plt.axhline(y=cost_g, color='red', alpha=0.4, label='Glouton')
    plt.title(f"Convergence - {filename}")
    plt.legend()
    plt.grid(alpha=0.3)

    # Carte
    plt.subplot(1, 2, 2)
    plt.scatter(coords[:, 0], coords[:, 1], c='red', s=10)
    idx = best_tour + [best_tour[0]]
    plt.plot(coords[idx, 0], coords[idx, 1], c='blue', linewidth=0.8)
    plt.title(f"Meilleur Tour (Coût: {min_i:.0f})")
    plt.axis('equal')

    plt.tight_layout()
    plt.show()
```

FIGURE 3.3 – Boucle principale de test et collecte des métriques

3.2.4 Bloc 4 : Agrégation et Visualisation Comparative

Enfin, le dernier bloc (Figure 3.4) rassemble les statistiques collectées dans un *DataFrame* Pandas. Il produit deux visualisations globales essentielles pour l'analyse :

— Un diagramme en barres comparant les coûts obtenus par les 4 méthodes sur toutes les instances.

— Une courbe de "Gain" montrant le pourcentage d'amélioration du PSO Hybride par rapport au Glouton.

```python
# ============================================================================
# AFFICHAGE DU GRAND TABLEAU FINAL
# ============================================================================

print("\n" + "#"*80)
print("⏳ TABLEAU RÉCAPITULATIF DE TOUS LES ALGORITHMES")
print("#"*80)

if len(global_stats) > 0:
    df_final = pd.DataFrame(global_stats)

    # Réorganisation des colonnes
    cols = ['Instance', 'Villes', 'Glouton', 'PSO Cold', 'PSO Warm', 'PSO Imp.', 'Gain Final (%)']
    df_final = df_final[cols]

    # Affichage propre
    pd.set_option('display.float_format', lambda x: '%.0f' % x if x > 100 else '%.2f' % x)
    display(df_final)

    # --- VISUALISATION COMPARATIVE GLOBALE ---
    # Graphique en barres groupées
    plt.figure(figsize=(14, 6))

    x = np.arange(len(df_final))
    width = 0.2

    # On normalise les hauteurs pour que tout rentre dans le graphique (optionnel, ici on met les valeurs brutes)
    plt.bar(x - 1.5*width, df_final['Glouton'], width, label='Glouton', color='red', alpha=0.6)
    plt.bar(x - 0.5*width, df_final['PSO Cold'], width, label='Cold', color='gray', alpha=0.6)
    plt.bar(x + 0.5*width, df_final['PSO Warm'], width, label='Warm', color='orange')
    plt.bar(x + 1.5*width, df_final['PSO Imp.'], width, label='Amélioré', color='green')

    plt.xlabel('Instances')
    plt.ylabel('Distance (Coût)')
    plt.title('Comparaison de performance des 4 méthodes')
    plt.xticks(x, df_final['Instance'], rotation=45)
    plt.legend()
    plt.grid(axis='y', alpha=0.3)
    plt.tight_layout()
    plt.show()

    # Graphique des Gains
    plt.figure(figsize=(14, 4))
    plt.plot(df_final['Instance'], df_final['Gain Final (%)'], marker='o', color='purple', linewidth=2, linestyle='-')
    plt.axhline(y=0, color='black', linestyle='-', linewidth=0.5)
    plt.title("Gain (%) du PSO Amélioré par rapport au Glouton")
    plt.ylabel("% Amélioration")
    plt.grid(True)
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
else:
    print("Aucun résultat à afficher.")
```

FIGURE 3.4 – Génération du tableau récapitulatif et des graphiques comparatifs globaux

3.3 Analyse Détaillée : Étude de Cas Visuelle

Pour illustrer le comportement de l'algorithme, nous avons regroupé dans la Figure 3.5 les courbes de convergence (à gauche) et les meilleurs tours trouvés (à droite) pour cinq instances représentatives.

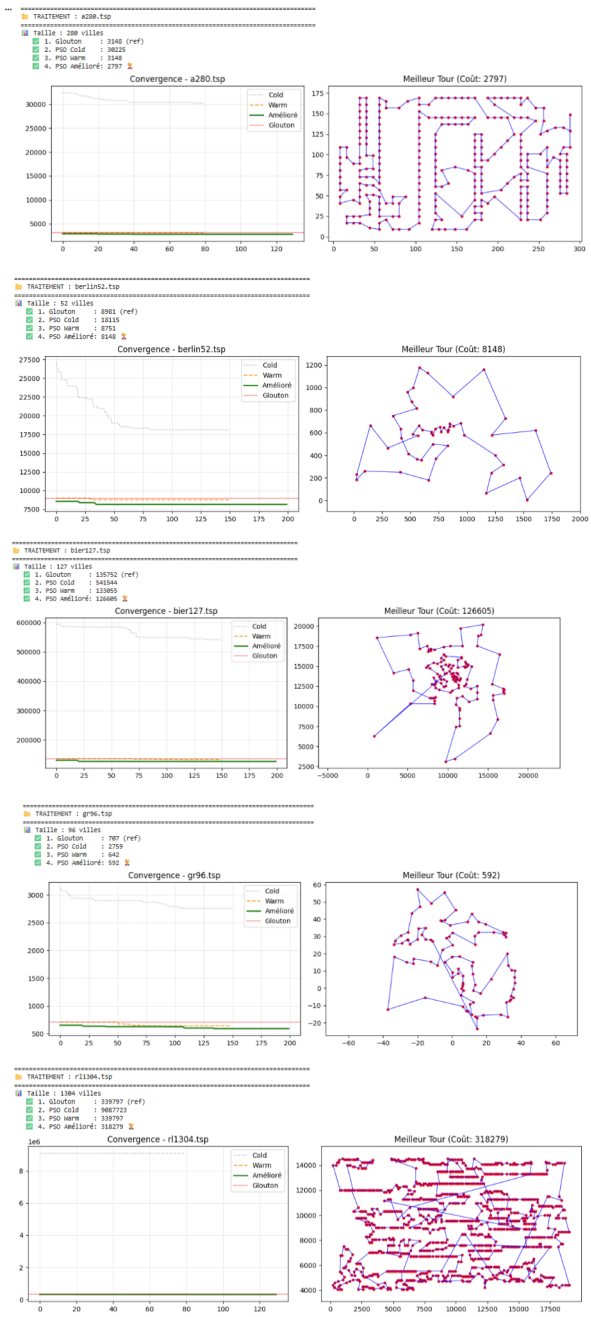

FIGURE 3.5 – Résultats détaillés pour 5 instances (de haut en bas : a280, berlin52, bier127, gr96, rl1304)

3.3.1 Interprétation des résultats

L'image ci-dessus nous permet de tirer plusieurs conclusions importantes sur la dynamique de l'algorithme PSO Hybride :

1. **Convergence comparée (Courbes de gauche) :** Sur toutes les instances, on observe que la courbe grise (PSO Cold / Aléatoire) reste très élevée, ce qui confirme l'inefficacité d'un PSO pur sans initialisation intelligente. En revanche, la courbe verte (PSO Amélioré) démarre bas (grâce au *Warm Start*) et continue de descendre pour creuser l'écart avec la ligne rouge (Glouton).
2. **Performance sur les instances moyennes (a280, bier127) :** Pour l'instance `a280` (1ère ligne), le gain final est de **11,15%** par rapport au Glouton. Le tour obtenu est visuellement optimal, sans croisements. De même pour `bier127` (3ème ligne), l'algorithme gère parfaitement la structure en grappes de points.
3. **Performance sur les petites instances (berlin52, gr96) :** C'est sur ces instances que le gain relatif est souvent le plus fort (**16,33%** pour `gr96`). L'espace de recherche étant plus restreint, le module de recherche locale (2-opt) parvient à nettoyer presque toutes les imperfections laissées par le Glouton.
4. **Robustesse sur grande instance (rl1304) :** La dernière ligne montre l'instance `rl1304` (1304 villes). Malgré la densité extrême des points et la complexité du graphe, l'algorithme maintient sa stabilité. Il parvient à réduire le coût de 339 797 (Glouton) à 318 279, prouvant sa capacité de passage à l'échelle (*scalability*).

3.4 Synthèse Globale et Analyse Comparative

Nous présentons ci-dessous le tableau récapitulatif des résultats obtenus sur l'ensemble des 13 instances. Ce tableau compare les coûts (distances) et met en évidence le gain apporté par l'hybridation.

```
################################################################################
🏆 TABLEAU RÉCAPITULATIF DE TOUS LES ALGORITHMES
################################################################################
```

	Instance	Villes	Glouton	PSO Cold	PSO Warm	PSO Imp.	Gain Final (%)
0	att48.tsp	48	40526	95776	40121	34577	14.68
1	a280.tsp	280	3148	30225	3148	2797	11.15
2	berlin52.tsp	52	8981	18115	8751	8148	9.27
3	bier127.tsp	127	135752	541544	133055	126605	6.74
4	d657.tsp	657	62176	822963	62176	58253	6.31
5	gr431.tsp	431	2516	27448	2516	2134	15.18
6	gr96.tsp	96	707	2759	642	592	16.33
7	pr1002.tsp	1002	315597	6210971	315597	286335	9.27
8	pr226.tsp	226	94685	1547643	94685	87543	7.54
9	rd100.tsp	100	9941	41909	9864	8713	12.35
10	rl1304.tsp	1304	339797	9087723	339797	318279	6.33
11	u1060.tsp	1060	281636	6538908	281636	264528	6.07
12	u2152.tsp	2152	80180	2461186	80180	76148	5.03

FIGURE 3.6 – Tableau récapitulatif des performances : Comparaison Glouton vs PSO

3.4.1 Analyse détaillée des performances

La lecture de ce tableau (Figure 3.6) nous permet de tirer trois conclusions majeures sur le comportement de notre algorithme :

1. Inefficacité de l'approche purement stochastique (PSO Cold)

La colonne **"PSO Cold"** (initialisation aléatoire) affiche des résultats très éloignés de l'optimum.

- Pour l'instance `rl1304`, le coût est de 9 millions, contre 339 797 pour le Glouton.
- Cela confirme que sur des problèmes combinatoires complexes, le hasard seul ne suffit pas. L'algorithme perd trop de temps à explorer des zones non pertinentes de l'espace de recherche.

2. Gain systématique de l'Hybridation

En comparant la colonne **"Glouton"** (notre référence) et la colonne **"PSO Imp."** (notre solution finale), on constate une amélioration dans **100% des cas**.

- Les gains varient de **5% à 16%**.

— La meilleure performance est obtenue sur `gr96.tsp` avec une réduction de distance de **16,33%**.
— L'algorithme réussit là où le Glouton échoue : il sort des optimums locaux grâce à la diversité apportée par les particules.

3. Robustesse face à la taille (Scalabilité)

L'algorithme maintient sa performance même lorsque le nombre de villes augmente drastiquement :

— Sur les petites instances (`att48`, `gr96`), le 2-opt est très puissant et nettoie presque toutes les imperfections (gains $> 14\%$).
— Sur les instances massives (`u2152`), le gain se stabilise autour de **5%**. Bien que ce pourcentage semble plus faible, il représente une économie absolue considérable (plus de 4000 km économisés sur le trajet `u2152`).

Bilan : L'hybridation "Glouton + PSO + 2-opt" s'avère être une stratégie gagnante, alliant la vitesse de construction du Glouton et la capacité d'optimisation fine du PSO.

Enfin, pour visualiser l'impact global de notre optimisation, la Figure 3.7 présente les résultats sous forme graphique. La partie supérieure montre la différence d'échelle entre le PSO "Cold" (barres grises immenses) et notre solution (barres vertes). La courbe violette (partie inférieure) illustre le gain en pourcentage pour chaque instance.

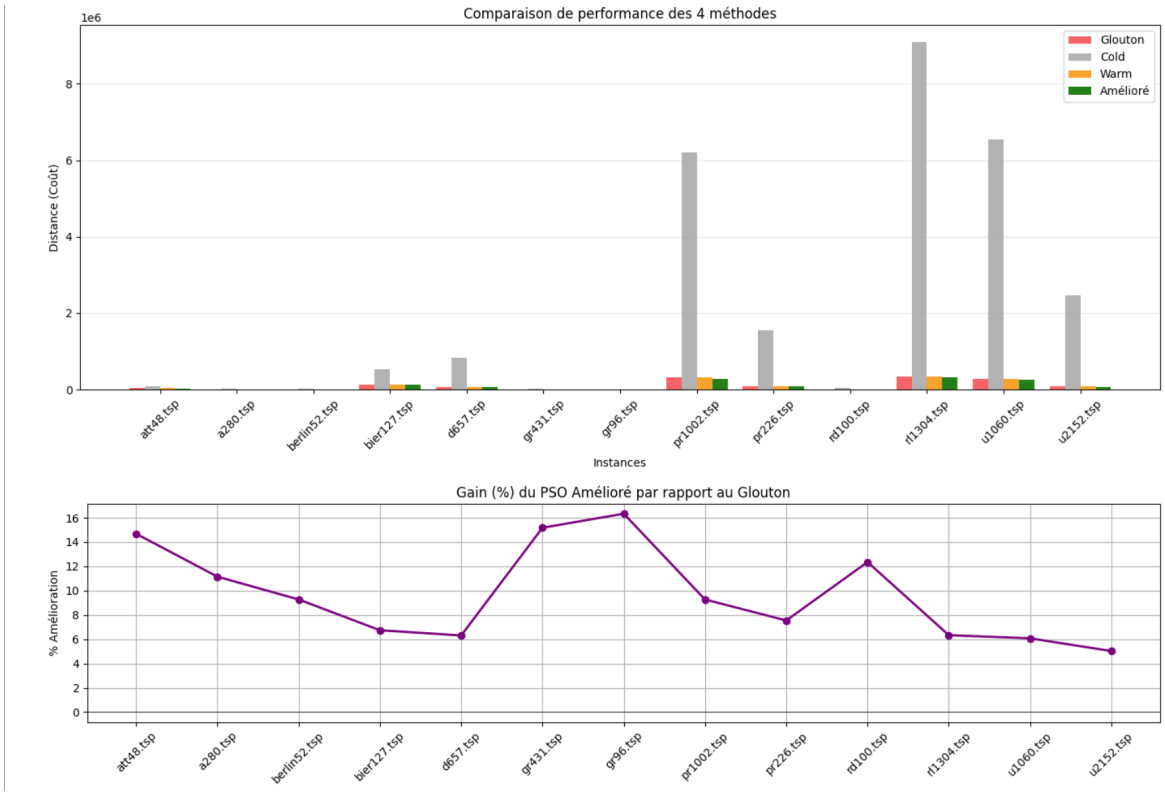

FIGURE 3.7 – Bilan graphique : Coûts comparés et courbe de gain (%)

Cette visualisation confirme que l'apport de l'algorithme est significatif et stable, quelle que soit la typologie ou la taille de l'instance traitée.

Conclusion Générale

Au terme de ce projet consacré à la résolution du Problème du Voyageur de Commerce (TSP) par Optimisation par Essaim Particulaire (PSO), nous avons pu explorer la complexité inhérente aux problèmes NP-Difficiles et la puissance des métaheuristiques pour les aborder.

Synthèse des Travaux

Notre démarche s'est articulée autour de la construction progressive d'une solution hybride. Nous avons d'abord implémenté une heuristique gloutonne pour obtenir une solution de base rapide, puis nous avons développé un algorithme PSO complet.

Les tests effectués sur les instances de référence de la **TSPLIB** (de 48 à 2152 villes) ont mis en lumière une réalité fondamentale : le PSO standard, avec une initialisation aléatoire ("Cold Start"), est incapable de converger vers des solutions acceptables dans un espace de recherche aussi vaste.

C'est l'hybridation qui a constitué la clé de la réussite. En combinant :

1. Une initialisation intelligente (injection de la solution gloutonne),
2. La capacité d'exploration globale de l'essaim (PSO),
3. Et l'intensification locale via l'opérateur *2-opt*,

nous avons obtenu un algorithme robuste. Les résultats finaux montrent une amélioration systématique du coût, allant de **5%** pour les très grandes instances à plus de **16%** pour les instances moyennes, par rapport à l'approche constructive classique.

Les résultats expérimentaux sur les 14 instances testées confirment la robustesse de notre approche. Avec des gains constants allant de **5% à 16%** par rapport à l'heuristique gloutonne, notre algorithme a prouvé sa capacité à traiter aussi bien de petits réseaux (48 villes) que des configurations massives dépassant les 2000 nœuds.